

Entrega #2 Mini-lenguaje

Jesús Ismael Ramos Recio

Cubo semántico

Se definen las reglas de inferencia de tipos para las operaciones del lenguaje. En todas las operaciones aritméticas, cuando se opera un valor entero (int) con un valor flotante (float), se promueve automáticamente a flotante. En el caso de los operadores relacionales, se devuelven un valor entero. El lenguaje maneja las variables booleanas de forma numérica, es decir, en lugar de “Verdadero” o “Falso”, se usará 1 para verdadero y 0 para falso. En la asignación de “int - float”, lo que sucede es que el valor se trunca y pierde decimales.

Operador	int - int	int - float	float - int	float - float
+	int	float	float	float
-	int	float	float	float
*	int	float	float	float
/	float	float	float	float
==	int	int	int	int
!=	int	int	int	int
>	int	int	int	int
<	int	int	int	int
>=	int	int	int	int
<=	int	int	int	int
=	int	int	float	float

Estructuras de datos utilizadas

En esta sección se definirá las estructuras de datos utilizadas para el Directorio de Funciones y la Tabla de Variables para el lenguaje.

Directorio de Funciones

Esta es una estructura que almacenará la información de las funciones declaradas en el programa. Para este lenguaje, se utilizará un diccionario para su implementación para poder buscar una función por su nombre en tiempo constante. El diccionario también es ideal para verificar duplicados y validar llamadas de forma rápida.

Estructura:

```
class DirectorioFunciones:
    # Estructura: Diccionario

    def __init__(self):
        self.funciones = {}

    def agregar(self, nombre, tipo_return, parametros):
        self.funciones[nombre] = {
            'tipo_return': tipo_return, # tipo de función
            'parametros': parametros,   # nombre de los parámetros
            'num_parametros': len(parametros) # num. de parámetros
        }

    def buscar(self, nombre):
        return self.funciones.get(nombre)

    def existe(self, nombre):
        return nombre in self.funciones
```

La estructura se puede ver dentro de la función “agregar”. Esta función se encarga de añadir una función al diccionario en el momento en que se declara una función. La función “buscar” obtiene información de una función para validar una llamada. Y la función “existe” está para verificar si ya fue declarada y evitar funciones duplicadas.

Tabla de Variables

La tabla de variables almacena la información de todas las variables del programa. Esta información está organizada por ámbitos, que son:

- Global: Una variable definida de forma global
- Funciones: Una variable definida dentro de una función

Para este lenguaje se maneja una Pila (Stack). la pila permite manejar ámbitos anidados. Entonces, cuando entra un nuevo ámbito, se apila un nuevo diccionario. Y al salir, todas las variables locales de dicho ámbito se eliminan automáticamente.

Estructura:

```
class TablaVariables:
    # Estructura: Pila de diccionarios
    # Forma de la estructura:
    #     pila_ambitos = [
    #         {},                                # Ámbito 0 (global)
    #         {"x": {...}},                      # Ámbito 1 (función)
    #         {"y": {...}}                       # Ámbito 2 (bloque dentro de función)
    #     ]

    def __init__(self):
```

```

self.pila_ambitos = [{}] # Ambito 0 = global
self.cont_ambitos = 0

def entrada_ambito(self):
    self.cont_ambitos += 1
    self.pila_ambitos.append({})

def salida_ambito(self):
    if len(self.pila_ambitos) > 1:
        self.pila_ambitos.pop()
        self.cont_ambitos -= 1

def declar_var(self, nombre, tipo, linea):
    ambito_act = self.pila_ambitos[-1]

    if nombre in ambito_act:
        print(f"Error semantico: Variable '{nombre}' ya declarada
en el mismo ambito en linea {linea}")
        return False

    ambito_act[nombre] = {
        'tipo': tipo,
        'inicializada': False,
        'ambito': self.cont_ambitos,
        'linea': linea
    }
    return True

def buscar_var(self, nombre):
    for i in range(len(self.pila_ambitos) - 1, -1, -1):
        if nombre in self.pila_ambitos[i]:
            return self.pila_ambitos[i][nombre]
    return None

```

Las variables se buscan desde el ámbito actual hacia el global, lo que permite ocultar variables de ámbitos superiores.

- La función “entrada_ambito()” hace ‘push’ a un nuevo ámbito cuando se define una función.
- La función “salida_ambito()” hace ‘pop’ al ámbito actual al salir de una función
- “declar_var(nombre, tipo, linea)” agrega una variable al ámbito actual.
- “buscar_var(nombre)” busca una variable desde el ámbito actual hacia el ámbito global.